# A Multi-Scale Rescaling Method for Ill-Conditioned Linear Systems

Lazizbek Sadullaev
Department of Mathematics and Statistics, Washington State University
Math 548 — Numerical Linear Algebra

**Abstract**

Classical iterative solvers such as Jacobi and Gauss–Seidel can converge slowly, or fail to converge at all, when the coefficient matrix of a linear system is ill-conditioned. In this report I study a column-norm rescaling technique — the *multi-scale method* of Wang and Wang [1] — that transforms an ill-conditioned system into an equivalent, better-conditioned one before an iterative method is applied. I derive the method from first principles, prove a bound relating the condition number of the rescaled system to that of the original system, and evaluate the method on two benchmark systems, comparing it against Jacobi and Gauss–Seidel in terms of iteration count and relative error.

## 1 Introduction

Solving a linear system

$$Ax = b, \qquad A \in \mathbb{R}^{n\times n} \text{ nonsingular}, \quad b \in \mathbb{R}^n, \tag{1}$$

is one of the most common subproblems in scientific computing. Two families of solvers exist: *direct* methods (e.g. LU decomposition), which compute an exact solution in a finite number of steps, and *iterative* methods (e.g. Jacobi, Gauss–Seidel, SOR, GMRES), which generate a sequence of approximations $x^{(0)}, x^{(1)}, x^{(2)}, \dots$ converging to $x$. Iterative methods are the standard choice for large, sparse systems, where a direct factorization is too costly to form or store.

The difficulty this report addresses is *ill-conditioning*. The sensitivity of the solution of (1) to small perturbations in $A$ or $b$ is governed by the condition number

$$\operatorname{cond}(A) = \|A\|\,\|A^{-1}\|, \tag{2}$$

which I take throughout in the spectral (2-)norm, so that $\operatorname{cond}_2(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$, the ratio of the largest to smallest singular value of $A$. When $\operatorname{cond}(A)$ is large, small floating-point errors can be amplified into large errors in the computed solution, and classical iterative methods frequently converge slowly or diverge outright. The multi-scale method I describe below does not change the underlying linear system; it rescales it into an equivalent problem with a smaller condition number, which can then be handed to any standard iterative solver.

## 2 Classical Iterative Methods

Both iterative methods used as a baseline in this report split $A = D + L + U$, where $D$ is the diagonal of $A$, and $L$, $U$ are its strictly lower- and upper-triangular parts.

**Jacobi method.** The Jacobi iteration uses only the diagonal part of $A$ to update every component simultaneously:

$$x^{(k+1)} = D^{-1}\big(b - (L+U)x^{(k)}\big). \tag{3}$$

**Gauss–Seidel method.** Gauss–Seidel uses the already-updated components within the same sweep:

$$x^{(k+1)} = (D + L)^{-1}\big(b - Ux^{(k)}\big). \tag{4}$$

Gauss–Seidel typically converges faster than Jacobi when both converge, because it uses the most recent information available at each step. Neither method, however, is guaranteed to converge for an arbitrary nonsingular $A$: convergence requires the iteration matrix's spectral radius to be less than 1, which fails for sufficiently ill-conditioned or poorly structured systems (this is demonstrated in §4.3).

# 3 The Multi-Scale Method

## 3.1 Construction

Let $d_1, d_2, \ldots, d_n$ denote the columns of $A$. Define the diagonal scaling matrix

$$D = \operatorname{diag}\left(\frac{1}{\|d_1\|_2}, \frac{1}{\|d_2\|_2}, \ldots, \frac{1}{\|d_n\|_2}\right), \tag{5}$$

i.e. the matrix that normalizes every column of $A$ to unit Euclidean length, and set

$$\widetilde{A} = AD. \tag{6}$$

Introducing the rescaled unknown $\widetilde{x} = D^{-1}x$, system (1) is equivalent to

$$\widetilde{A}\,\widetilde{x} = b. \tag{7}$$

Because $D$ is diagonal and invertible, (7) has exactly the same solution set as (1) once undone by the linear map $x = D\widetilde{x}$; nothing about the problem has changed except the scale on which each unknown is measured. The idea is that many ill-conditioned systems arise precisely because different columns of $A$ (i.e. different variables' influence) live on wildly different numerical scales — normalizing those scales away can dramatically shrink the condition number, even though the underlying problem is untouched.

## 3.2 Algorithm

**Step 1.** Compute the column 2-norms $\|d_1\|_2, \ldots, \|d_n\|_2$ of $A$ and form $D = \operatorname{diag}(1/\|d_j\|_2)$.

**Step 2.** Form the rescaled matrix $\widetilde{A} = AD$ (the right-hand side $b$ is unchanged).

**Step 3.** Solve $\widetilde{A}\widetilde{x} = b$ with any iterative method (Jacobi, Gauss–Seidel, GMRES, ...).

**Step 4.** Recover the solution to the original system by undoing the scaling: $x = D\widetilde{x}$.

## 3.3 A Guaranteed Bound on the Condition Number

The following result, proved originally by Wang and Wang [1], shows that this rescaling can never make the conditioning *worse*, and typically makes it substantially better.

**Theorem 1.** *Let $d_{\max} = \max_j \|d_j\|_2$ and $d_{\min} = \min_j \|d_j\|_2$ over the columns of $A$. Then*

$$\operatorname{cond}(\widetilde{A}) \;\le\; \frac{d_{\max}}{d_{\min}}\,\operatorname{cond}(A). \tag{8}$$

*Proof.* Since $\widetilde{A} = AD$, submultiplicativity of the induced matrix norm gives

$$\operatorname{cond}(\widetilde{A}) = \|\widetilde{A}\|\,\|\widetilde{A}^{-1}\| = \|AD\|\,\|D^{-1}A^{-1}\| \le \|A\|\,\|D\|\,\|D^{-1}\|\,\|A^{-1}\|. \tag{9}$$

Because $D$ is diagonal with entries $1/\|d_j\|_2$, its induced 2-norm is its largest entry in absolute value,

$$\|D\| = \max_j \frac{1}{\|d_j\|_2} = \frac{1}{d_{\min}}, \tag{10}$$

and similarly $D^{-1} = \operatorname{diag}(\|d_j\|_2)$ has

$$\|D^{-1}\| = \max_j \|d_j\|_2 = d_{\max}. \tag{11}$$

Substituting back,

$$\operatorname{cond}(\widetilde{A}) \le \|A\|\,\|A^{-1}\| \cdot \frac{1}{d_{\min}} \cdot d_{\max} = \frac{d_{\max}}{d_{\min}} \operatorname{cond}(A). \tag{12}$$

□

**Remark 1.** *When every column of $A$ already has the same Euclidean length, $d_{\max} = d_{\min}$ and $D$ is (up to a scalar) orthogonal, in which case (8) reduces to $\operatorname{cond}(\widetilde{A}) \le \operatorname{cond}(A)$: rescaling can only help. The bound (8) is loosest — and the method most useful — precisely when $A$'s columns are badly mismatched in scale, which is a common source of ill-conditioning in practice (e.g. physical systems where variables are recorded in different units).*

# 4 Numerical Experiments

Both examples below follow the same protocol: I solve the system directly with `numpy.linalg.solve` to obtain the exact solution $x_{\text{exact}}$, then run Jacobi, Gauss–Seidel, and the multi-scale method (rescaling followed by Gauss–Seidel), each starting from $x^{(0)} = 0$ with tolerance $10^{-10}$ and a cap of 1000 iterations. Convergence is measured by iteration count (IT) and the relative error $\|x^{(k)} - x_{\text{exact}}\|_2$ (ERR).

## 4.1 Implementation

The three solvers and the condition-number utilities used throughout are given below.

Listing 1: Jacobi, Gauss–Seidel, and multi-scale solvers.

```python
import numpy as np

def jacobi(A, b, x0, tol=1e-10, max_iterations=1000):
    x = x0.copy()
    D = np.diag(A)
    R = A - np.diagflat(D)
    for i in range(max_iterations):
        x_new = (b - np.dot(R, x)) / D
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            break
        x = x_new
    return x, i + 1

def gauss_seidel(A, b, x0=None, tol=1e-10, max_iterations=1000):
    n = len(b)
    x = np.zeros(n) if x0 is None else x0.copy()
    D = np.diag(np.diag(A))
    L = np.tril(A, -1)
    U = np.triu(A, 1)
```

```
    for k in range(max_iterations):
        x_new = np.linalg.solve(D + L, b - np.dot(U, x))
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            return x_new, k
        x = x_new
    return x, max_iterations

def multi_scale(A, b, tol=1e-10, max_iterations=1000):
    d = np.linalg.norm(A, axis=0)       # column 2-norms
    D = np.diag(d)
    A_tilde = A @ D                      # rescaled matrix
    x0 = np.zeros_like(b, dtype=float)
    x_tilde, num_it = gauss_seidel(A_tilde, b, x0=x0, tol=tol,
                                   max_iterations=max_iterations)
    x = D @ x_tilde                      # undo the scaling
    return x, num_it

def condition_number(A):
    sv = np.linalg.svd(A, compute_uv=False)
    return sv.max() / sv.min(), sv
```

### 4.2 Example 1: A Badly Scaled System

Consider

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 100 & 1 \\ 1 & 0 & 10000 \end{pmatrix}, \qquad b = \begin{pmatrix} 2 \\ 101 \\ 10001 \end{pmatrix}, \qquad x_{\text{exact}} = (1,\, 1,\, 1)^{\top}. \tag{13}$$

The entries of $A$ span four orders of magnitude, giving $\text{cond}_2(A) \approx 1.00 \times 10^4$, with singular values $\sigma(A) \approx \{10000.0,\ 100.0,\ 1.0\}$ — almost singular. Rescaling each column to unit norm (Step 1–2 of §3) gives

$$\widetilde{A} = AD \approx \begin{pmatrix} 0.7071 & 0.0100 & 0 \\ 0 & 0.9999 & 0.0001 \\ 0.7071 & 0 & 0.9999 \end{pmatrix}, \tag{14}$$

with singular values $\sigma(\widetilde{A}) \approx \{1.307,\ 1.000,\ 0.541\}$, giving $\text{cond}_2(\widetilde{A}) \approx 2.41$. This is a reduction of nearly four orders of magnitude, consistent with the bound of Theorem 1 since the columns of $A$ are extremely mismatched in scale ($d_{\max}/d_{\min}$ is large here).

Table 1: Results for Example 1.

| Method | Iterations | Relative Error |
|---|---|---|
| Jacobi | 7 | $1.73 \times 10^{-12}$ |
| Gauss–Seidel | 5 | $9.99 \times 10^{-15}$ |
| Multi-scale (+ Gauss–Seidel) | 5 | $9.77 \times 10^{-15}$ |

Here Gauss–Seidel already converges quickly on the unscaled system, so the multi-scale method's main contribution is confirming the theoretical prediction $\text{cond}(\widetilde{A}) \le (d_{\max}/d_{\min})\,\text{cond}(A)$: it drives the condition number down to nearly its orthogonal-scaling floor without changing the iteration count or the (already excellent) accuracy.

### 4.3 Example 2: An Intrinsically Ill-Conditioned System

Consider the Hilbert-like system

$$A = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}, \qquad b = (1,\, 1,\, 1)^\top, \qquad x_{\text{exact}} = (3,\, -24,\, 30)^\top. \tag{15}$$

Here $\text{cond}_2(A) \approx 524.1$, and rescaling gives $\text{cond}_2(\widetilde{A}) \approx 373.7$ — a much more modest improvement than in Example 1.

Table 2: Results for Example 2 (iteration cap of 1000 reached in all cases).

| Method | Iterations | Relative Error |
|---|---|---|
| Jacobi | 1000 (diverged) | $\infty$ |
| Gauss–Seidel | 1000 | $1.65 \times 10^{-7}$ |
| Multi-scale (+ Gauss–Seidel) | 1000 | $1.65 \times 10^{-7}$ |

Jacobi's iterates blow up (components reach $\mathcal{O}(10^{236})$), since the iteration matrix's spectral radius exceeds 1 for this system; Gauss–Seidel converges but has not reached the $10^{-10}$ tolerance within 1000 iterations, and the multi-scale variant performs essentially identically.

### 4.4 Discussion

The two examples illustrate an important limitation of the method: the size of the improvement in (8) is governed by the ratio $d_{\max}/d_{\min}$ of the column norms, so multi-scale rescaling helps most when a matrix's poor conditioning is caused by *mismatched variable scales* (Example 1, where entries span $10^0$ to $10^4$). When ill-conditioning is instead *intrinsic* to the matrix's structure — as in Example 2, a near-singular Hilbert-type matrix whose columns are already comparable in norm but nearly linearly dependent — column normalization has little to work with, and the condition number improves only modestly ($524.1 \to 373.7$), leaving the iteration count and error essentially unchanged. This matches the bound of Theorem 1: when $d_{\max}/d_{\min}$ is close to 1, rescaling is guaranteed to help only marginally.

## 5 Conclusion

The multi-scale method is a lightweight preprocessing step: form $\widetilde{A} = AD$ by normalizing the columns of $A$, solve the rescaled system with any iterative method, and undo the scaling. I proved that this transformation can never increase the condition number by more than the factor $d_{\max}/d_{\min}$, and confirmed numerically that it is highly effective when ill-conditioning stems from scale mismatch across columns (a $10^4 \to 2.4$ reduction in Example 1), but offers only limited benefit when ill-conditioning is intrinsic to the matrix's structure rather than its scaling (Example 2). In both cases the method is essentially free to apply and never makes an iterative solver's job harder, which argues for using it as a default preprocessing step ahead of Jacobi- or Gauss–Seidel-type solvers.

## References

[1] J. Wang and K. Wang, *Multi-scale method for ill-conditioned linear systems*, IOP Conference Series: Materials Science and Engineering, vol. 242, p. 012100, 2017. https://doi.org/10.1088/1757-899X/242/1/012100

[2] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, Baltimore, MD, 2013.

[3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, Philadelphia, PA, 2003.